MuktoKormo | Weekly Progress Report

Bidding System, Milestone Delivery Workspace, Role Dashboards, Supabase Backend, Database Schema & Authentication

Project	MuktoKormo (Freelance Marketplace)	Team	Abdul Ahad Akid (2207018) | Ashraful Hosen (2207042)
Frontend	Next.js 16, React 19, Tailwind v4	Backend	Supabase (PostgreSQL, Auth, RLS)
Scope	Frontend workflows + Database & Authentication wiring	Next Phase	Live Data Wiring & Session Layer

1. Executive Summary & Architectural Status

MuktoKormo is an Upwork/Fiverr-grade freelance marketplace engineered for the Bangladeshi digital ecosystem. It connects local employers with verified remote talent through a milestone-based escrow workflow, pairing Upwork-caliber interface mechanics with localized financial rails (BDT Escrow, bKash, Nagad). In this reporting period the platform moved from mockup interfaces to a feature-complete frontend, and was then taken off its mock foundation and connected to a real hosted Supabase backend.

Key milestones: (1) Interactive Freelancer Bidding & Proposal Evaluation System; (2) Contract & Milestone Delivery Workspace with immutable audit trails; (3) Dedicated Client and Freelancer Role Dashboards with distinct financial ledgers; (4) a production PostgreSQL schema deployed from the specification's ER diagram with Row Level Security on every table; (5) /login and /signup wired to Supabase Auth.

Architecture: The presentation layer (Next.js 16 / React 19 / Tailwind v4) is unchanged. The backend is Supabase - managed PostgreSQL with PostgREST auto-APIs, GoTrue authentication (v2.197) and per-row RLS. The hosted project is live (ref dhkhwaazyzikaoaontzn); the publishable API key was validated at runtime against the auth and PostgREST endpoints.

2. Platform Feature Deliverables

2.1 Interactive Freelancer Bidding & Proposal System

- **Proposal submission (BidModal.tsx):** bid amount in BDT, delivery days, structured milestones, cover letter and portfolio references; the flat 7% platform fee and net earnings are calculated in real time.
- **Localized payout selector:** bKash, Nagad, Rocket or direct Bank Transfer, with account number binding on bid submission.
- **Employer evaluation room (JobBidsModal.tsx):** side-by-side comparator showing NID verification status, review ratings, completed jobs, milestone payment breakdowns and cover letters.
- **One-click actions:** shortlist a candidate, or 'Accept Bid & Lock Escrow' which opens the escrow milestone funding modal with pre-filled pricing.
- **'My Bids' drawer (MyBidsDrawer.tsx):** tracks 'Pending', 'Shortlisted' and 'Accepted' proposals, with one-click withdrawal and navbar badge count sync.

2.2 Contract & Milestone Delivery Workspace (/contracts, /contracts/[id])

- **Milestone pipeline:** Escrow Funded -> In Progress -> Submitted for Review -> Approved & Released, with status badges and completion meters.
- **Deliverable submission:** work notes, live demo/preview links (GitHub PRs, Figma, staging URLs) and simulated file attachments.
- **Review & escrow release:** clients inspect work, request revisions with feedback, or approve in one click to release escrow funds to the freelancer's wallet.
- **Immutable audit log:** every event (funding, submissions, review requests, releases, platform fees) logged with actor and relative timestamps in English and Bengali.
- **Financial ledger sidebar:** total commitment, funds in escrow, funds released and remaining deliverables.

2.3 Role-Based Dashboards (/dashboard/client, /dashboard/freelancer)

- **Client dashboard:** job post manager, proposal inbox with candidate metrics, talent discovery, saved talent, escrow expenditure overview and a 4-step verification checklist (Email, Phone, Billing, NID).
- **Freelancer dashboard:** available balance, pending escrow, lifetime earnings, MFS Cashout Modal (bKash/Nagad calculator), 'Recommended Jobs' feed with instant bid triggers and a profile strength meter.
- **Unified shell (DashboardShell.tsx):** collapsible sidebar, persistent notifications and a role switcher between hiring and freelancing modes.

3. Supabase Backend & Database Schema

- **Provisioning:** hosted project created; publishable key stored in .env.local (git-ignored) with a committable .env.example. Only two dependencies added: @supabase/supabase-js (2.117.2) and @supabase/ssr.

- **Client factories:** src/lib/supabase/client.ts (browser) and src/lib/supabase/server.ts (Server Components / route handlers, async cookies() API).
- **Environment validation:** key decodes and matches the project ref, /auth/v1/health returns 200, and PostgREST responds (rejecting only absent tables).
- **Migration:** supabase/migrations/20261007000000_initial_schema.sql (48 statements), validated with a PostgreSQL grammar (pglast) before being applied. It creates seven tables as diagrammed, plus indexes, constraints and the signup trigger:

Table	Purpose	Key Columns	Relations & Integrity
users	Identity directory (1:1 with auth.users)	id (uuid PK), email UNIQUE, name, created_at	id FK -> auth.users.id; created by handle_new_user trigger
clients	Employer company profile	id, user_id, company_name, location	user_id FK UNIQUE -> users.id (1:1)
freelancers	Freelancer professional profile	id, user_id, headline, bio, skills text[], hourly_rate numeric(12,2)	user_id FK UNIQUE -> users.id (1:1)
projects	Job postings by clients	id, client_id, title, description, category, budget_min/max, timeline_days, status	client_id FK -> clients.id; status CHECK (draft/open/in_progress/completed/archived); budget & date checks
proposals	Freelancer offers	id, project_id, freelancer_id, cover_letter, proposed_amount, timeline_days, status	FK -> projects.id; UNIQUE (project_id, freelancer_id); status CHECK
contracts	Agreed engagement (escrow)	id, proposal_id, total_amount, milestone_count, status, signed_at	proposal_id FK UNIQUE (1:1); status CHECK (pending/active/completed/cancelled)
reviews	Post-contract feedback	id, contract_id, reviewer_id, reviewee_id, rating, comment	contract_id FK UNIQUE (1:1); rating CHECK 1-5

Design notes: money is numeric(12,2) to avoid float drift in escrow math; every status field has a CHECK constraint; all foreign keys are indexed; 1:1 relationships use UNIQUE foreign keys. Passwords are owned by Supabase Auth (bcrypt in auth.users.encrypted_password), never plaintext or exposed via the API; a handle_new_user trigger creates the users row (email + full name) on every signup.

Row Level Security: enabled on all seven tables and proven live - anonymous reads return zero rows and anonymous writes fail with error 42501.

Role	Tables	Permitted operations
anon	All seven	None. Reads return zero rows; writes rejected (42501). Tables closed by default.
authenticated	users	Select / insert / update own row only (auth.uid() = id).
authenticated	clients, projects	Directory reads of client profiles; create / update / delete own projects.
authenticated	freelancers, proposals	Directory reads of freelancer profiles; create / update / withdraw own proposals.
authenticated (party-scoped)	contracts, reviews	Access resolved via joins to auth.uid() - only the project owner (client) or proposal owner (freelancer) may act.

4. Authentication

Login (/login)

- Server-side verification via supabase.auth.signInWithPassword; session persisted through @supabase/ssr cookie storage and survives reloads.
- Role-aware redirect preserved: users land on the Client or Freelancer dashboard based on their profile row.
- Password reset via resetPasswordForEmail with an in-page confirmation note and error banner; Forgot? link retained.
- SMS-OTP and Google/Apple options removed this week (they need external provider credentials); the surface is now email & password only, with OAuth/OTP tracked as future work.

Signup (/signup)

- Both flows call supabase.auth.signUp with raw_user_meta_data.full_name; the trigger fills users.name automatically.
- **Freelancer:** 4-step wizard (role, talent profile, payment, submit) ends with signUp and a freelancers row insert (headline, hourly rate, skills, bio).
- **Client:** new 'Your account' step (name / email / password / terms), then signUp and a clients row (company name).
- Email-confirmation aware: confirmation screen when 'Confirm email' is on, direct dashboard redirect with auto-confirm - one code path for both. Passwords are hashed by GoTrue (bcrypt) and never touched by application code.

5. Verification & Quality Assurance

- **End-to-end:** a disposable signup created the auth user, the trigger created the public.users row with correct email and name, a session was returned, own-row reads/updates worked, and a self-scoped clients insert was accepted.

- **Security:** cross-tenant reads were suppressed (the user saw only their own row); the anonymous write probe returned 42501 on all seven tables; all constraints and indexes confirmed on the live database.
- **Credentials:** project ref matches the key, health endpoint 200, PostgREST reachable, login/signup flows complete with the real anon key.
- **Build quality:** tsc --noEmit clean on all new files; next build compiles and prerenders every route; ESLint's 21 findings are all pre-existing legacy components, none from new code; no regressions in public routes, dashboards or the post-job flow.

6. Work in Progress & Next Sprint Objectives

- Add middleware (proxy.ts) for server-side session refresh and a sign-out control with @supabase/ssr.
- Replace dashboard, feed and job-post mock data with live Supabase queries behind existing component contracts.
- Introduce a role/admin concept, live project and proposal creation, and dispute and messaging persistence.
- Provision Storage buckets for KYC and portfolio files; enable Realtime for proposals and in-chat messages.
- Integrate bKash and Nagad Checkout APIs (sandbox) for webhook-confirmed escrow locking.
- Real-time messaging inside the Contract Delivery Workspace (Supabase Realtime / WebSocket).
- NID OCR verification pipeline with liveness detection to elevate freelancer trust badges.
- Re-introduce SMS-OTP login (SMS provider needed) and Google/Apple OAuth (provider console setup needed).